SCHOOL OF COMPUTING AND DATA SCIENCES
FLAME UNIVERSITY
PUNE - 412115, MAHARASHTRA, INDIA

A Staged TCP/IP-Inspired Transport Protocol Implementation over UDP in C

A Graduation Project

Submitted by

PRATIK PATIL

For the award of the degree

Of

BSC. COMPUTER SCIENCE AND DESIGN

April 2026

THESIS CERTIFICATE

This is to undertake that the Graduation Project titled **A STAGED TCP/IP-INSPIRED TRANSPORT PROTOCOL IMPLEMENTATION OVER UDP IN C**, submitted by me to the School of Computing and Data Sciences, FLAME University, for the award of **BSc. Computer Science and Design**, is a bonafide record of the research work done by me under the supervision of **Dr. Aamod Sane**. The contents of this Graduation Project, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

In order to effectively convey the ideas presented in this Graduation Project, all protocol diagrams, comparative figures, and analytical tables in the main report were created by the author specifically for this submission.

Standard technical references cited in the bibliography are used only for academic context and conceptual grounding. No copyrighted third-party figure is reproduced in the main body of the report.

Pune - 412115, Maharashtra, India **Pratik Patil**

Date: April 2026

Dr. Aamod Sane
School of Computing and Data Sciences
FLAME University

© 2026 FLAME University

ACKNOWLEDGEMENTS

This graduation project would not have been possible without the generous support and guidance of several individuals and institutions. I would like to extend my sincere gratitude to all who supported me during this work.

First and foremost, I sincerely thank **Dr. Aamod Sane**, my research advisor, for his invaluable guidance, constructive feedback, and continuous encouragement throughout this project. His direction helped me understand transport-layer design with clarity and improve both the technical depth and presentation quality of this thesis.

I am grateful to the **School of Computing and Data Sciences, FLAME University**, for providing a strong academic environment, useful learning resources, and institutional support needed to carry out this work.

I also thank my peers and colleagues for their insightful discussions, implementation suggestions, and moral support during development, debugging, and report preparation.

Finally, I express heartfelt thanks to my family for their patience, understanding, and constant motivation. Their support gave me the confidence and focus required to complete this graduation project successfully.

ABSTRACT

KEYWORDS TCP/IP; UDP sockets; reliability; checksum; sliding window; selective repeat; adaptive timeout; congestion control; fast retransmit; fast recovery

This report presents the design and staged implementation of a TCP/IP-inspired communication stack built over UDP sockets in C. The project contains twenty- five sender-receiver pairs (from level 0 to level 24) and demonstrates a clear evolution from basic bit transfer to advanced transport behavior similar to real TCP principles. Early levels focus on message encoding and reconstruction. Middle levels add parity checking, acknowledgements, timeout-based retries, and stop-and-wait reliability. Later levels introduce connection setup and teardown, packet checksum, sequence and acknowledgement numbers, pipelining, sliding windows, chunk-based payload transfer, out-of-order buffering, and adaptive retransmission timeout (RTO) estimation.

The full project context also includes the expanded web visualizer that covers versions 0 to 24. In that interface, the code, finite state machine, and UML timeline are shown together so that the protocol progression can be studied as an educational sequence instead of as isolated source files.

An important objective of this project is learning networking from scratch through a textbook-first approach. The conceptual foundation follows standard sources such as the top-down networking method of Kurose and Ross and the implementation perspective of Stevens. AI tools were used only in a responsible support role for idea exploration, clarification, and debugging direction. The final implementation logic, coding, and validation were performed manually, without copying generated code.

The final levels implement congestion control ideas such as slow start, congestion avoidance, and, in the most advanced version, fast retransmit and fast recovery based on

duplicate acknowledgements. Even though this project is an educational simplification of TCP and uses UDP as the underlying transport, it captures many practical protocol engineering concepts: correctness, efficiency, robustness to packet loss and reordering, and maintainable protocol state management.

The educational value is especially important. The combination of versioned C code, textbook references, and a synchronized visualizer makes TCP-like behavior easier to understand for novice students who often find transport protocols and congestion control difficult at first glance.

The report explains the architecture, algorithms, implementation decisions, version-by-version progression, and limitations. It also provides diagrams, comparison tables, and formula-based analysis for throughput and timeout logic. The outcome is a complete learning artifact that bridges networking theory with hands-on systems programming.

CONTENTS

		Page
ACKNOWLEDGEMENTS		**i**
ABSTRACT		**ii**
LIST OF TABLES		**vii**
LIST OF FIGURES		**viii**
CHAPTER 1	**INTRODUCTION**	**1**
1.1	Problem Statement	2
1.2	Project Objectives	3
1.3	System Context and Setup	4
1.4	Architecture Overview	4
1.5	Version Progression Strategy	5
1.6	Version-Wise Reference Backbone	5
1.7	Educational Value	5
1.8	Scope and Boundaries	7
1.9	Research Questions	7
1.10	Report Organization	8
CHAPTER 2	**PROTOCOL DESIGN AND IMPLEMENTATION**	**9**
2.1	Global Design Principles	9
2.2	Version Family A: Bit and Byte Transfer (v0–v2)	10
2.2.1	Version 0: Single-Bit UDP Demo	10
2.2.2	Version 1: Character as 8 Independent Bits	11
2.2.3	Version 2: Full String by Bit Stream	12
2.3	Version Family B: Error Detection and Stop-and-Wait (v3–v7)	13
2.3.1	Version 3: Parity Detection	13
2.3.2	Version 4: Sequence Numbers	14
2.3.3	Version 5: Stop-and-Wait ARQ with ACK/NAK	15
2.3.4	Version 6: Timeout and Retries	16
2.3.5	Version 7: Struct-Based Packetization	17
2.4	Version Family C: TCP-like Framing and Pipelining (v8–v10)	18
2.4.1	Version 8: Handshake, Flags, and Connection Lifecycle	18
2.4.2	Version 9: Sliding Window (Go-Back-N Style)	19
2.4.3	Version 10: Chunk-Based Payloads	20
2.5	Version Family D: Selective Repeat and Adaptive Control (v11–v14)	21
2.5.1	Version 11: Selective-Repeat-Like Buffering	21
2.5.2	Version 12: Adaptive RTO Estimation	22
2.5.3	Version 13: Congestion Window and Slow Start	23
2.5.4	Version 14: Fast Retransmit and Fast Recovery	24

2.6 Version Family E: Adaptive and Reno-Like Extensions (v15–v24) . . . 25
2.6.1 Version 15: RTT-Smoothed Timeout Refinement 26
2.6.2 Version 16: Adaptive Window Discipline 27
2.6.3 Version 17: Timestamp Option Integration 28
2.6.4 Version 18: SACK-Inspired Receiver Reporting 29
2.6.5 Version 19: Congestion-Avoidance Stabilization 30
2.6.6 Version 20: Loss-Recovery Coupled with Option Feedback 31
2.6.7 Version 21: Sender State Consolidation 32
2.6.8 Version 22: Receiver Reassembly and Delivery Hardening 33
2.6.9 Version 23: End-to-End Robustness Tuning 34
2.6.10 Version 24: Final Integrated Transport Stage 35
2.6.11 Family E Summary 36
2.7 Chapter Summary 36

CHAPTER 3 ALGORITHMIC AND QUANTITATIVE ANALYSIS 37

3.1 Bit Encoding and Message Cost 37
3.2 Parity-Based Error Detection 37
3.3 Stop-and-Wait Service Time 38
3.4 Checksum in TCP-like Versions 38
3.5 Sliding Window Efficiency 39
3.6 Chunking Gain 39
3.7 Adaptive RTO Estimation 40
3.8 Congestion Window Dynamics 40
3.8.1 Slow Start 40
3.8.2 Congestion Avoidance 40
3.8.3 Loss Response 41
3.9 Latency Components 41
3.10 Reliability Metrics 42
3.11 A Practical Theorem-like Observation 42
3.12 Interpretation for This Project 43

CHAPTER 4 ILLUSTRATIONS, COMPARATIVE ANALYSIS, AND FINAL OUTCOMES 44

4.1 End-to-End Protocol Lifecycle 44
4.2 Version Capability Layers 45
4.3 Reliability Progress Chart 46
4.4 Throughput-Oriented Improvement Chart 47
4.5 Comparative Feature Matrix 48
4.6 Loss-Recovery Behavior Illustration 49
4.7 Key Findings 49
4.8 Limitations 49
4.9 Conclusion 50

APPENDIX A EXTENDED TECHNICAL APPENDIX 51

A.1 Appendix Purpose 51

A.2 Complete Build and Run Workflow 51
A.2.1 Compile 51
A.2.2 Execute 52
A.2.3 Operational Notes 52
A.3 Debug Checklist Used in This Project 52
A.4 Detailed Behavior: Sender Core Loop 53
A.4.1 State Variables 53
A.5 Detailed Behavior: Receiver Core Loop 54
A.5.1 Receiver Buffering in Selective-Repeat Style 54
A.6 Version-by-Version Pitfall Notes 54
A.6.1 v0–v2 Pitfalls 54
A.6.2 v3–v7 Pitfalls 54
A.6.3 v8–v10 Pitfalls 55
A.6.4 v11–v14 Pitfalls 55
A.7 Experiment Design Template 55
A.8 Sample Observation Narrative 56
A.9 Illustrative State Machine View 56
A.10 Protocol Trace Formatting Recommendation 56
A.11 Suggested Quantitative Extensions 57
A.12 Security and Reliability Perspective 57
A.13 Code Organization Recommendations 58
A.14 Reproducibility Notes 58
A.15 Expanded Reflection on Learning Outcomes 58
A.15.1 Networking Concepts 58
A.15.2 Systems Programming 59
A.15.3 Algorithmic Thinking 59
A.15.4 Technical Communication 59
A.16 Expanded Testing Scenarios and Interpretation 59
A.17 Evaluation Rubric for Academic Assessment 59
A.18 Final Reflection 61
A.19 Appendix Summary 61

BIBLIOGRAPHY **62**

LIST OF TABLES

Table	Caption	Page
1.1	Reference backbone for the staged protocol families.	6
4.1	Feature comparison across key protocol milestones.	48
A.1	Standard experiment template used for each protocol version.	55
A.2	Suggested assessment rubric for implementation quality.	60

LIST OF FIGURES

Figure	Caption	Page
1.1	High-level architecture of the sender-receiver protocol system over UDP.	4
2.1	Version 0 architecture: single-bit UDP transmission from sender to receiver.	10
2.2	Version 1 architecture: one character transmitted as eight UDP bit-datagrams and reconstructed at receiver side.	11
2.3	Version 2 architecture: full string transmitted as a bit stream with NULL-terminated completion.	12
2.4	Version 3 architecture: parity-based corruption detection over UDP payload transfer.	13
2.5	Version 4 architecture: sequence-numbered UDP units for order visibility and reliability groundwork.	14
2.6	Version 5 architecture: stop-and-wait ARQ with ACK/NAK feedback path.	15
2.7	Version 6 architecture: timeout-driven retransmission with duplicate-safe receiver handling.	16
2.8	Version 7 architecture: struct-based packetization with unified sender and receiver parsing flow.	17
2.9	Version 8 architecture: handshake flags, lifecycle control, and checksum-protected data flow.	18
2.10	Version 9 architecture: sliding window pipelining with cumulative ACK and timeout-based retransmission.	19
2.11	Version 10 architecture: chunk-based payload transfer with in-window buffering and ordered reassembly.	20
2.12	Version 11 architecture: selective-repeat-like buffering with out-of-order storage and in-order delivery.	21
2.13	Version 12 architecture: RTT sampling with adaptive RTO control for variable-delay conditions.	22
2.14	Version 13 architecture: congestion window growth, ACK clocking, and loss-triggered rate reduction.	23
2.15	Version 14 architecture: duplicate-ACK loss inference with fast retransmit and fast recovery control.	24
2.16	Version 15 architecture: smoothed RTT-driven retransmission timeout refinement.	26
2.17	Version 16 architecture: timeout-aware adaptive window control.	27
2.18	Version 17 architecture: timestamp insertion and echo-aware feedback loop.	28
2.19	Version 18 architecture: SACK-inspired gap reporting and selective resend planning.	29
2.20	Version 19 architecture: stabilized congestion-avoidance behavior with ACK-clocked feedback.	30

2.21 Version 20 architecture: fast recovery guided by richer feedback signals. 31
2.22 Version 21 architecture: consolidated sender control for stable decision flow. 32
2.23 Version 22 architecture: hardened receiver reassembly and contiguous delivery control. 33
2.24 Version 23 architecture: coordinated end-to-end robustness tuning under mixed failures. 34
2.25 Version 24 architecture: final integrated transport design across reliability, timing, and recovery. 35

4.1 Complete session lifecycle used by advanced protocol versions. 44
4.2 Layered capability growth across protocol version families. 45
4.3 Illustrative reliability trend as protocol features mature. 46
4.4 Illustrative throughput improvement from staged protocol evolution. . . 47
4.5 Simplified fast-loss-recovery event path in advanced versions. 49

A.1 Simplified sender-side operational state flow in advanced versions. . . . 56
A.2 Recommended structured protocol-trace fields for automated analysis. . 56

CHAPTER 1

INTRODUCTION

Reliable communication is a central problem in computer networks. At first look, sending data from one machine to another may appear simple. In practice, real communication must handle packet loss, duplication, corruption, delay variation, and out-of-order delivery. Transport protocols such as TCP were designed to solve these issues. However, the behavior of TCP can feel abstract when studied only through theory.

This project addresses that learning gap through a staged implementation of transport protocol features over UDP sockets in C. Instead of directly using the operating system's TCP stack, the project builds many TCP-like ideas in application code. This approach makes each design decision visible. The sender and receiver logic, sequence management, timeout handling, and congestion responses are explicitly implemented and observed.

The full work contains twenty-five sender-receiver versions, from level 0 to level 24. The earliest levels implement bit transmission and byte reconstruction. The middle levels introduce error detection and stop-and-wait reliability. The advanced levels include handshake, checksum, sliding window, selective-repeat style buffering, adaptive timeout estimation, timestamp and SACK-like ideas, and congestion control mechanisms that resemble TCP Reno behavior.

1.1 PROBLEM STATEMENT

The project aims to answer one practical question:

> How can we incrementally construct a reliable and adaptive transport protocol over unreliable UDP datagrams while keeping the implementation understandable for learning and experimentation, while also using AI tools responsibly for idea support (not code copying) and validating every design step through standard networking textbooks?

UDP provides only best-effort datagram delivery. It does not guarantee packet arrival, packet order, duplicate suppression, congestion control, or connection state. Therefore, the application layer must implement these properties if they are required. This project demonstrates that process step by step. The learning method in this work is book-first and concept-first: AI tools were used only to discuss alternatives, debugging direction, and pseudocode ideas, while final logic, code writing, and verification were done manually.

The conceptual path follows a top-down networking approach in the spirit of widely used teaching references, especially Kurose and Ross for protocol layering and transport intuition, Stevens for implementation-level thinking about TCP/IP behavior, and Tanenbaum and Wetherall for broader networking foundations Kurose and Ross (2021); Stevens and Wright (1995); Tanenbaum and Wetherall (2011); Postel (1981); Paxson *et al.* (2011); Allman *et al.* (2009); Mathis *et al.* (1996); Borman *et al.* (2014); Postel (1980); Hall (2024).

1.2 PROJECT OBJECTIVES

The major objectives of the project are listed below.

1. Build a complete progression from raw transmission to advanced transport control.

2. Keep each version self-contained, so that learners can compare behavior across levels.

3. Follow a responsible learning workflow: study textbook concepts first, then use AI tools only for idea exploration, and finally implement and test independently without copying generated code.

4. Implement reliability features incrementally: parity, ACK/NAK, timeout retries, sequence control, and retransmission logic.

5. Move from one-byte packets to chunk-based payload transfer to improve throughput.

6. Introduce adaptive timing and congestion-aware sending to emulate practical transport design.

7. Document implementation, assumptions, limitations, and future improvements in a formal technical report.

1.3 SYSTEM CONTEXT AND SETUP

All implementations use IPv4 UDP sockets (`AF_INET` with `SOCK_DGRAM`) on port 9090. The sender transmits to localhost (`127.0.0.1`), and the receiver binds to `INADDR_ANY`. Typical message size in this project is bounded to around 99 characters.

The same high-level execution pattern is followed in all versions:

1. Start receiver.
2. Start sender.
3. Enter message in sender terminal.
4. Observe transfer, ACK behavior, retries, and completion.

1.4 ARCHITECTURE OVERVIEW

Figure 1.1 shows the conceptual architecture. Even though the implementation is local (single host), the control logic is equivalent to a networked deployment.

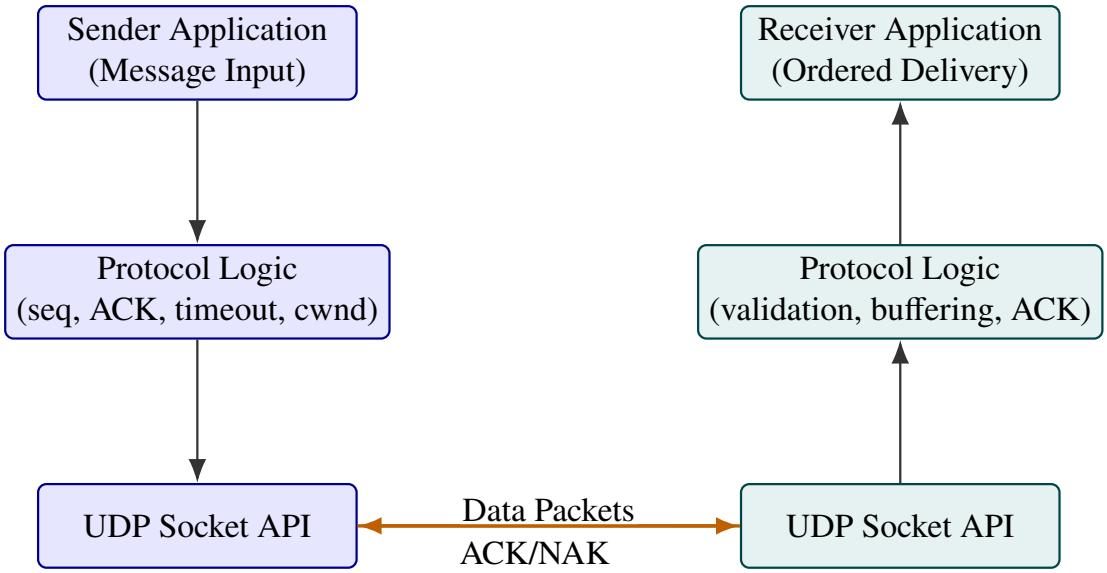

Figure 1.1: High-level architecture of the sender-receiver protocol system over UDP.

1.5 VERSION PROGRESSION STRATEGY

The project does not attempt to implement all features at once. Instead, it follows a layered progression model:

1. **Foundation phase (v0–v2):** bit and byte transfer.
2. **Reliability phase (v3–v7):** parity, sequencing, stop-and-wait, timeout retries, packet struct design.
3. **Transport phase (v8–v14):** handshake, checksum, connection close, sliding window, chunking.
4. **Adaptive and Reno-like phase (v15–v24):** adaptive RTO, timestamp/SACK-inspired refinements, congestion control, fast retransmit, and recovery-oriented behavior.

1.6 VERSION-WISE REFERENCE BACKBONE

Each version family is grounded in a small set of reference sources that shaped its design and implementation style.

This phased model provides two major advantages. First, debugging remains manageable because each level introduces a small set of new behaviors. Second, the relationship between protocol features and observed performance becomes clear in experiments.

1.7 EDUCATIONAL VALUE

From an academic perspective, this project provides a practical bridge between computer networks, systems programming, and algorithm design. Core concepts from textbooks become concrete:

1. **Data integrity:** parity and checksum validation.
2. **Reliability:** acknowledgement and retransmission.
3. **Ordering:** sequence numbers and expected sequence tracking.
4. **Pipelining:** multiple in-flight packets via windows.

Version Family	Main Ideas	Reference Backbone and Code Inspiration
v0–v2	UDP bit/byte transfer, reconstruction, end-of-message handling	RFC 768 for UDP, Beej's Guide for socket programming basics, Stevens for implementation discipline, Kurose and Ross for protocol layering Postel (1980); Hall (2024); Stevens and Wright (1995); Kurose and Ross (2021)
v3–v7	Parity, sequence awareness, ACK/NAK, stop-and-wait reliability	Kurose and Ross for reliable transport concepts, Tanenbaum and Wetherall for error detection and protocol reasoning, Stevens for practical C socket implementation Kurose and Ross (2021); Tanenbaum and Wetherall (2011); Stevens and Wright (1995)
v8–v14	Handshake, checksum, sliding window, selective-repeat style buffering, adaptive timeout, fast retransmit	RFC 793, RFC 6298, Kurose and Ross, and Stevens for transport semantics and timer design Postel (1981); Paxson *et al.* (2011); Kurose and Ross (2021); Stevens and Wright (1995)
v15–v24	Adaptive control, RTT tuning, timestamps, SACK-like ideas, congestion avoidance, educational Reno-style recovery	RFC 5681, RFC 2018, RFC 7323, RFC 6298, RFC 793, Stevens, and Kurose and Ross Allman *et al.* (2009); Mathis *et al.* (1996); Borman *et al.* (2014); Paxson *et al.* (2011); Postel (1981); Stevens and Wright (1995); Kurose and Ross (2021)

Table 1.1: Reference backbone for the staged protocol families.

5. **Flow under uncertainty:** adaptive timeout and congestion response.

Because each feature is implemented manually, the learner sees trade-offs directly. For example, stop-and-wait is easy to reason about but inefficient at higher latency. Sliding windows improve throughput but increase state complexity. Selective-repeat buffering improves robustness under reordering but requires careful bookkeeping.

The interactive visualizer further strengthens educational value by showing the sender code, receiver code, finite state machine, and UML timeline together. That makes protocol state transitions easier to follow for beginners who are learning transport concepts for the first time.

1.8 SCOPE AND BOUNDARIES

The project is intentionally scoped for learning and controlled experiments. Important boundaries are listed below.

1. Communication is primarily localhost based.
2. Address and port are hardcoded in most versions.
3. Security features such as encryption and authentication are not included.
4. Receiver-advertised window and full SACK format are not fully implemented.
5. Most versions handle one sender-receiver flow at a time.

These boundaries are acceptable for a graduation project focused on protocol mechanics. They also leave clear directions for future extension.

1.9 RESEARCH QUESTIONS

The implementation and analysis are organized around practical questions:

1. How much reliability can be achieved using only user-space logic over UDP?
2. At which stages does throughput improve significantly, and why?
3. How does adaptive timeout outperform fixed timeout under variable delays?
4. How does congestion window control affect retransmission behavior and completion time?
5. How can students use AI tools responsibly for protocol learning (idea-level guidance and debugging support) while preserving original implementation ownership and textbook-grounded correctness?

1.10 REPORT ORGANIZATION

The remainder of this report is structured as follows.

1. Chapter 2 describes protocol design and version-wise implementation details from the foundation phase to the advanced control phase.

2. Chapter 3 presents algorithmic foundations and quantitative analysis with equations.

3. Chapter 4 provides illustrations, result-oriented tables, practical observations, and final conclusions.

This structure keeps the narrative simple: design first, analysis second, evaluation and insights third.

CHAPTER 2

PROTOCOL DESIGN AND IMPLEMENTATION

This chapter presents the full version-by-version protocol evolution. The implementation strategy is incremental. Each new version keeps previous capabilities and introduces one or more carefully selected features. This approach improves correctness and makes debugging practical.

2.1 GLOBAL DESIGN PRINCIPLES

The same design principles are followed across all versions.

1. **Single responsibility per stage:** each version focuses on a specific protocol concept.

2. **Explicit state:** sender and receiver maintain clearly named variables for sequence numbers, timeouts, and window state.

3. **Deterministic transitions:** packet receipt leads to a defined state update path.

4. **Traceable behavior:** console output helps validate the protocol timeline in each test run.

2.2 VERSION FAMILY A: BIT AND BYTE TRANSFER (V0–V2)

The first family introduces communication mechanics at the lowest useful level.

2.2.1 Version 0: Single-Bit UDP Demo

In level 0, the sender sends one bit to the receiver using a UDP datagram. There is no framing beyond the single value. This version validates:

1. socket creation,
2. destination addressing,
3. datagram transmission,
4. datagram reception.

This step is intentionally minimal and prepares the path for structured packet design.

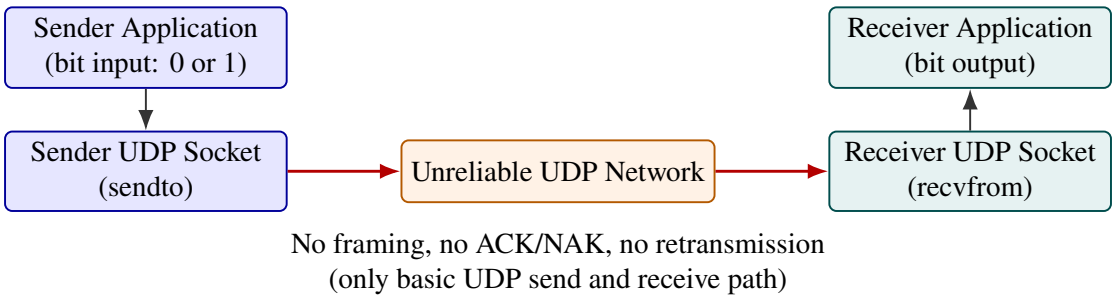

Figure 2.1: Version 0 architecture: single-bit UDP transmission from sender to receiver.

2.2.2 Version 1: Character as 8 Independent Bits

In level 1, one character is encoded into 8 bits and sent as 8 UDP datagrams. The receiver reconstructs the byte by shifting and OR operations. Timing is regulated using small sleep intervals so packet order remains easier to track in local runs.

Educational value:

1. understanding of binary representation,
2. significance of bit order,
3. reconstruction logic at receiver side.

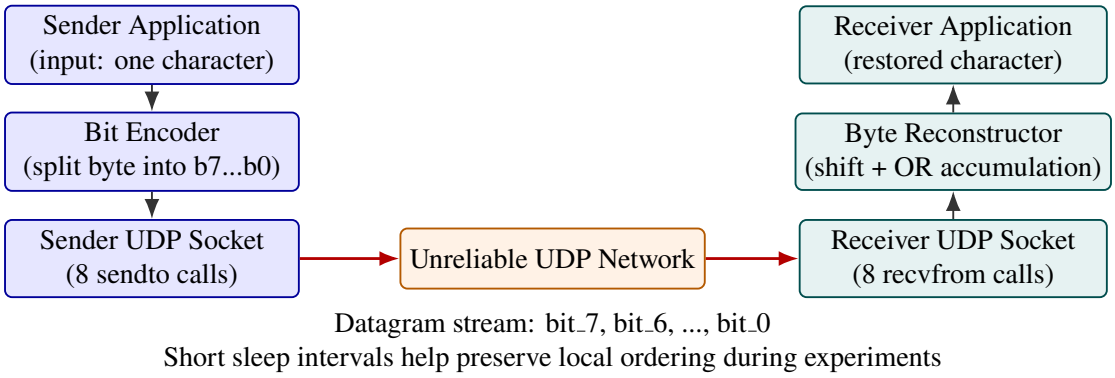

Figure 2.2: Version 1 architecture: one character transmitted as eight UDP bit-datagrams and reconstructed at receiver side.

2.2.3 Version 2: Full String by Bit Stream

Level 2 extends the bit-stream approach to full strings. A null character is sent at the end to mark message completion. At this stage, communication works for complete user input, but no error detection or reliability control exists.

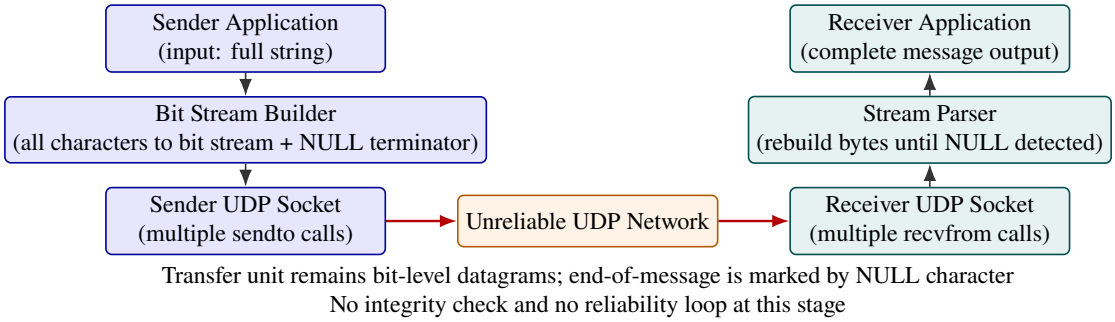

Figure 2.3: Version 2 architecture: full string transmitted as a bit stream with NULL-terminated completion.

2.3 VERSION FAMILY B: ERROR DETECTION AND STOP-AND-WAIT (V3–V7)

Family B introduces reliability fundamentals.

2.3.1 Version 3: Parity Detection

For each byte, sender computes parity and transmits it with payload bits. Receiver recomputes parity and compares values.

Outcome:

1. corruption can be detected,
2. corruption cannot yet be corrected.

Parity is simple and fast but weak against many multi-bit error patterns.

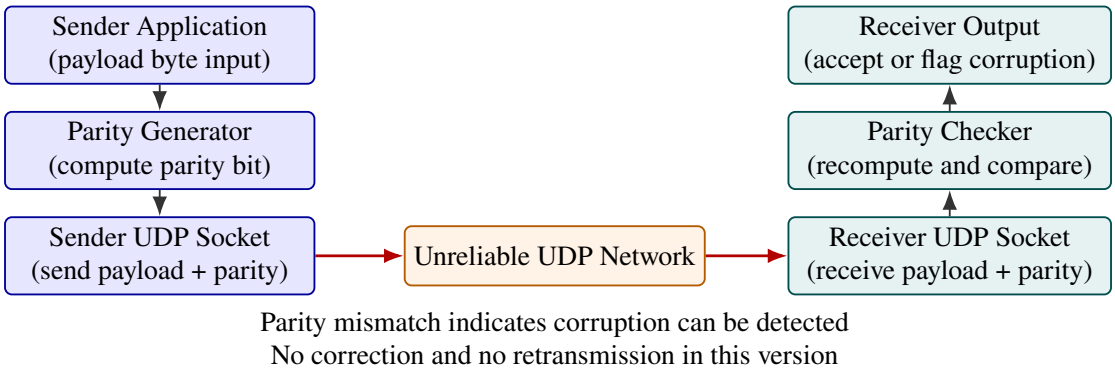

Figure 2.4: Version 3 architecture: parity-based corruption detection over UDP payload transfer.

2.3.2 Version 4: Sequence Numbers

Level 4 attaches sequence numbers to per-character transfer. This improves observability and creates the basis for reliable acknowledgement logic.

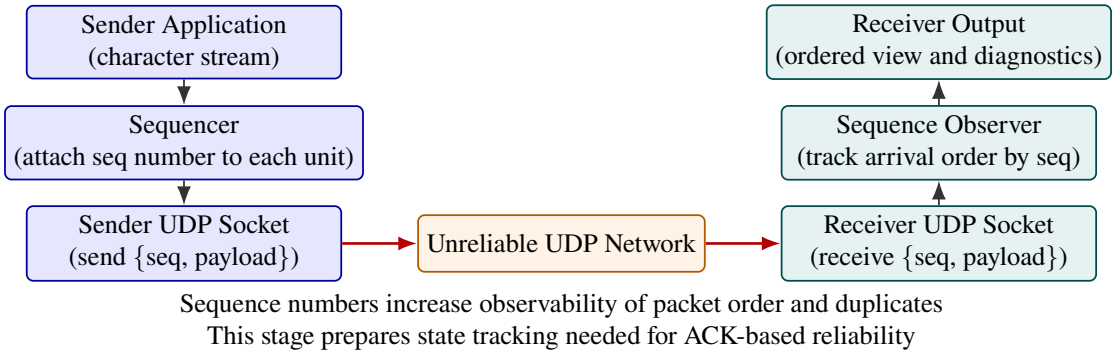

Figure 2.5: Version 4 architecture: sequence-numbered UDP units for order visibility and reliability groundwork.

2.3.3 Version 5: Stop-and-Wait ARQ with ACK/NAK

In level 5, receiver sends ACK on success and NAK on parity failure. Sender transmits one unit and waits for response before moving forward. This is the first true reliability loop.

Strength:

1. simple correctness argument,
2. easy handling of duplicates,
3. straightforward debugging.

Limitation:

1. low throughput because channel stays idle while waiting.

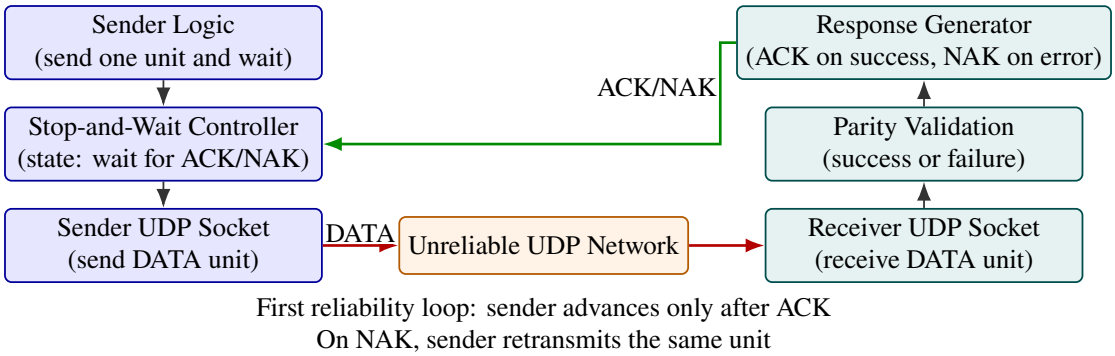

Figure 2.6: Version 5 architecture: stop-and-wait ARQ with ACK/NAK feedback path.

2.3.4 Version 6: Timeout and Retries

Level 6 adds socket receive timeout and retry limit. If ACK does not arrive in time, sender retransmits. Receiver tracks expected sequence and handles duplicates conservatively.

This version is the first practically robust design under packet loss.

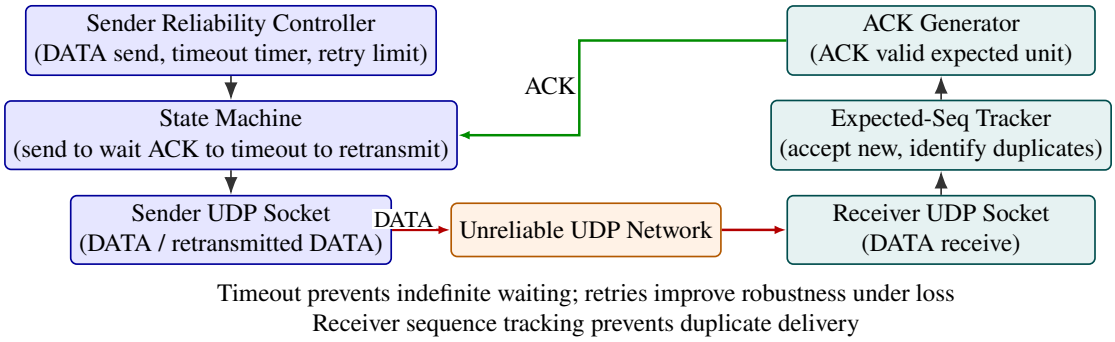

Figure 2.7: Version 6 architecture: timeout-driven retransmission with duplicate-safe receiver handling.

2.3.5 Version 7: Struct-Based Packetization

Version 7 replaces loose fields with an explicit packet structure:

1. sequence number,
2. payload,
3. parity,
4. end flag.

This change improves code readability and prepares the architecture for multi-field TCP-like packets in later levels.

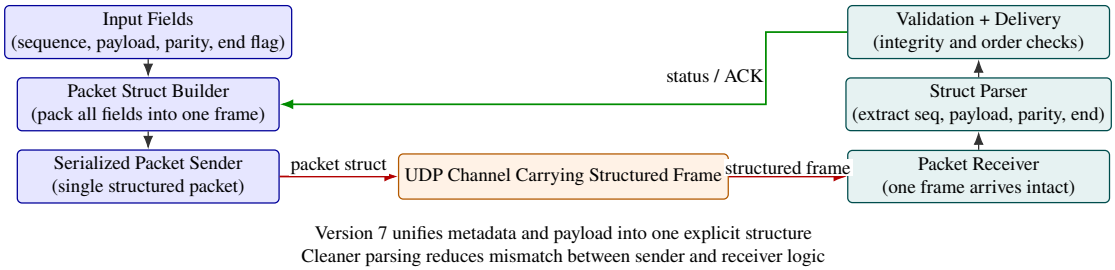

Figure 2.8: Version 7 architecture: struct-based packetization with unified sender and receiver parsing flow.

2.4 VERSION FAMILY C: TCP-LIKE FRAMING AND PIPELINING (V8–V10)

This family introduces transport semantics closer to TCP.

2.4.1 Version 8: Handshake, Flags, and Connection Lifecycle

Level 8 includes:

1. SYN for start,
2. ACK for acknowledgement,
3. DATA for payload,
4. FIN for graceful close.

The sender and receiver perform a simplified three-way handshake and teardown. Checksum is also introduced at this level for stronger integrity checks.

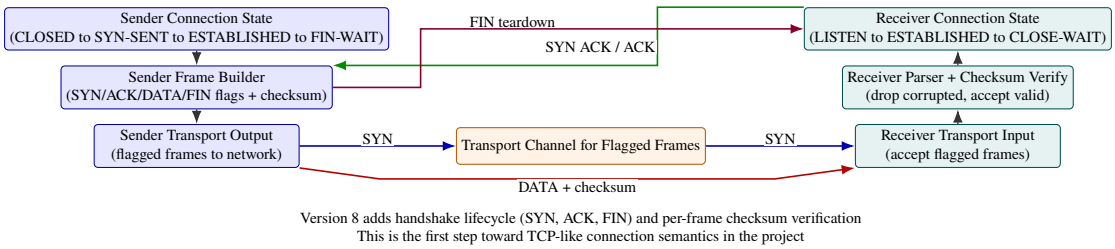

Figure 2.9: Version 8 architecture: handshake flags, lifecycle control, and checksum-protected data flow.

2.4.2 Version 9: Sliding Window (Go-Back-N Style)

Version 9 increases efficiency with a sender window (typical size 4). Sender may transmit multiple packets before waiting for ACKs. On timeout, packets from base are retransmitted. Receiver sends cumulative-like ACK behavior based on expected sequence.

This level marks a major throughput jump because waiting time is amortized.

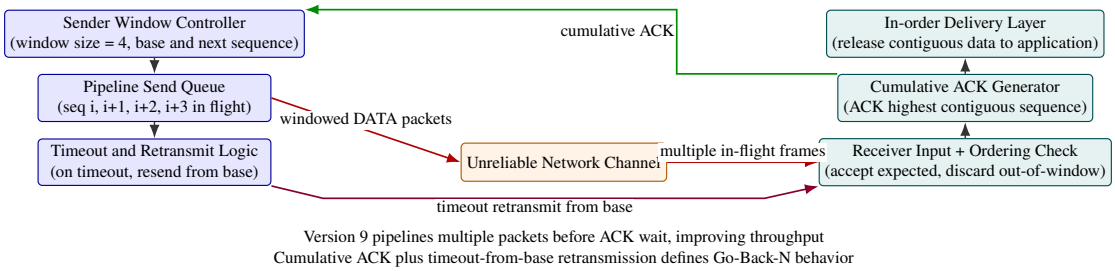

Figure 2.10: Version 9 architecture: sliding window pipelining with cumulative ACK and timeout-based retransmission.

2.4.3 Version 10: Chunk-Based Payloads

Version 10 increases data per packet by introducing `CHUNK_SIZE = 8` bytes. Benefits:

1. fewer packets for same message length,
2. less per-packet header overhead,
3. better throughput for moderate messages.

Receiver logic starts supporting in-window buffering, enabling transition to selective-repeat style delivery in the next family.

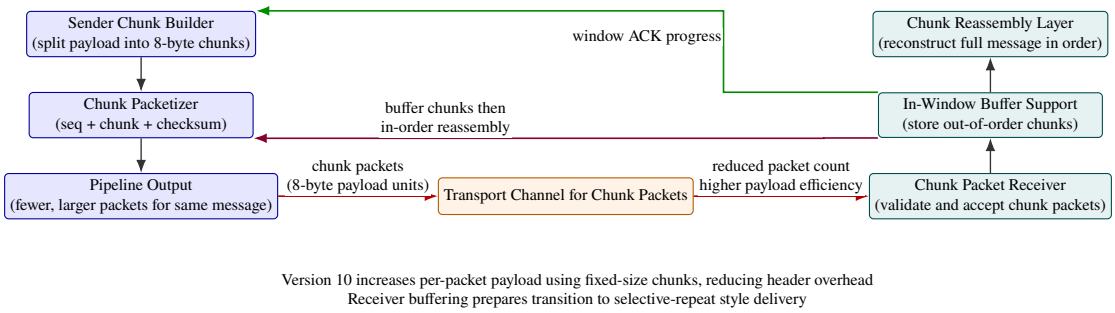

Figure 2.11: Version 10 architecture: chunk-based payload transfer with in-window buffering and ordered reassembly.

2.5 VERSION FAMILY D: SELECTIVE REPEAT AND ADAPTIVE CONTROL (V11–V14)

Family D contains the most advanced protocol behavior in this project.

2.5.1 Version 11: Selective-Repeat-Like Buffering

Receiver stores out-of-order chunks in buffers and delivers data in order when missing sequence arrives. This avoids unnecessary retransmission of already received data and improves behavior under reordering.

Key receiver structures include:

1. packet-received flags,
2. chunk data buffer,
3. chunk length metadata,
4. expected sequence tracker.

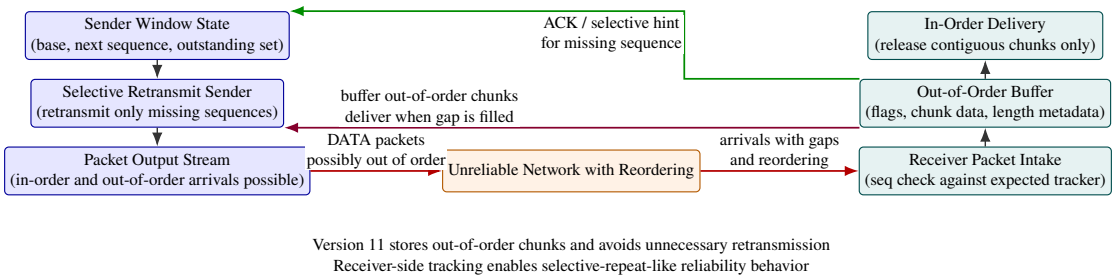

Figure 2.12: Version 11 architecture: selective-repeat-like buffering with out-of-order storage and in-order delivery.

2.5.2 Version 12: Adaptive RTO Estimation

Version 12 introduces RTT sampling and adaptive RTO calculation. Fixed timeout works poorly in variable-delay conditions. Adaptive timeout adjusts retransmit threshold dynamically and reduces both premature retransmits and long idle waits.

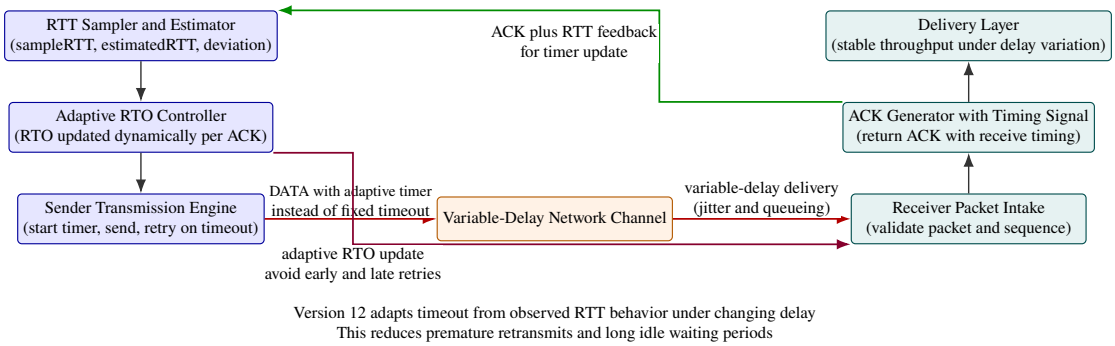

Figure 2.13: Version 12 architecture: RTT sampling with adaptive RTO control for variable-delay conditions.

2.5.3 Version 13: Congestion Window and Slow Start

This level introduces congestion control variables:

1. `cwnd` (congestion window),
2. `ssthresh` (slow start threshold).

Behavior follows a simplified policy:

1. grow quickly in slow start,
2. grow gradually in congestion avoidance,
3. reduce sending rate after loss indication.

This improves fairness and stability compared to aggressive fixed-window sending.

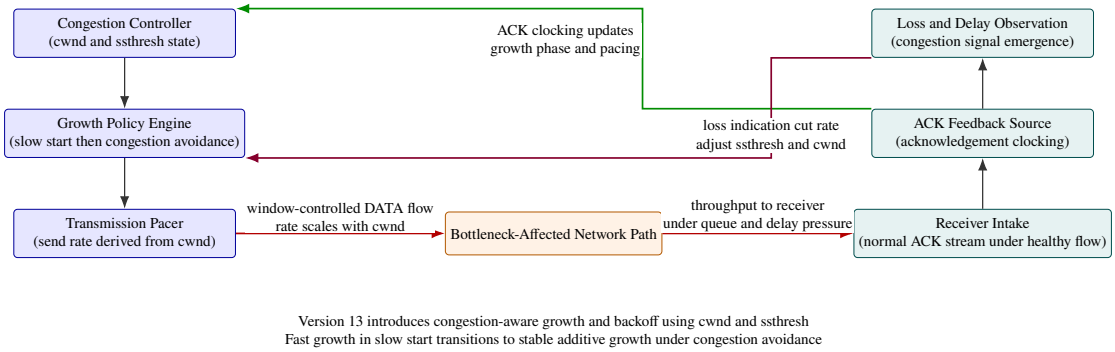

Figure 2.14: Version 13 architecture: congestion window growth, ACK clocking, and loss-triggered rate reduction.

2.5.4 Version 14: Fast Retransmit and Fast Recovery

Version 14 is the most complete implementation within Family D. The project then continues through versions 15–24, which extend this baseline with more advanced adaptive and Reno-like refinements. Enhancements include:

1. duplicate ACK counting,
2. triple-duplicate-ACK loss inference,
3. fast retransmission without waiting for timeout,
4. temporary fast-recovery window inflation and controlled deflation.

This behavior closely resembles a learning-scale version of TCP Reno.

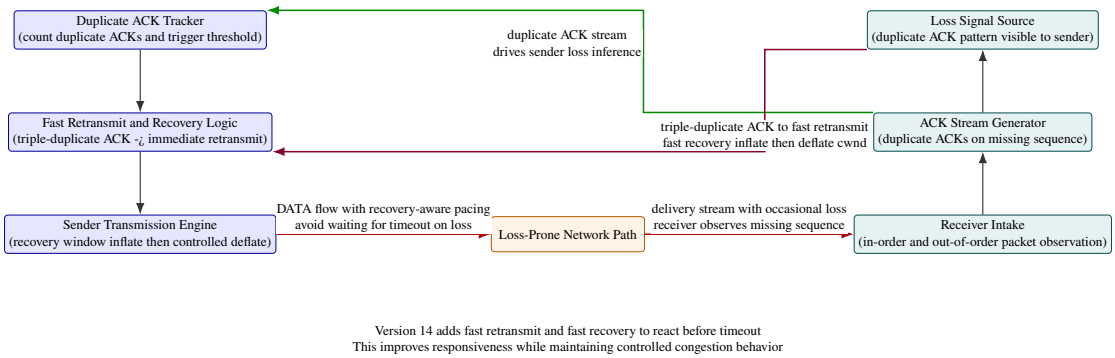

Figure 2.15: Version 14 architecture: duplicate-ACK loss inference with fast retransmit and fast recovery control.

2.6 VERSION FAMILY E: ADAPTIVE AND RENO-LIKE EXTENSIONS (V15–V24)

The later versions extend the same transport story into a more advanced and educational TCP-like space. This family focuses on RTT-sensitive timeout refinement, timestamp-oriented state awareness, SACK-like reasoning, congestion-avoidance behavior, and improved recovery handling. The visualizer also uses these later versions to show how the FSM and UML timeline can be used to explain protocol decisions step by step.

The primary design inspiration for this family comes from the RFCs and books that describe TCP evolution in practice. In particular, RFC 6298 explains the retransmission timer, RFC 5681 covers congestion control behavior, RFC 2018 describes selective acknowledgment options, RFC 7323 covers TCP extensions for high performance, and RFC 793 provides the core TCP model. The implementation style is also informed by Stevens, Kurose and Ross, and Tanenbaum and Wetherall, because those texts help connect protocol theory with the concrete sender and receiver code structure used in the project Paxson *et al.* (2011); Allman *et al.* (2009); Mathis *et al.* (1996); Borman *et al.* (2014); Postel (1981); Stevens and Wright (1995); Kurose and Ross (2021); Tanenbaum and Wetherall (2011).

This family is especially valuable in the report because it shows the point at which the learning project becomes visually and conceptually close to real TCP thinking, while still remaining simple enough for students to trace manually.

2.6.1 Version 15: RTT-Smoothed Timeout Refinement

Version 15 stabilizes retransmission timing using smoothed RTT and deviation tracking. Instead of relying on coarse timeout behavior, this version updates RTO with a more disciplined estimator so sender retries are less noisy.

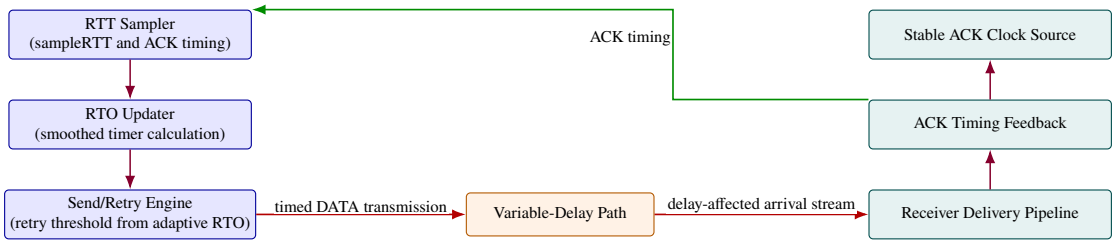

Figure 2.16: Version 15 architecture: smoothed RTT-driven retransmission timeout refinement.

2.6.2 Version 16: Adaptive Window Discipline

Version 16 connects timeout behavior with sender pacing and window discipline. When timing uncertainty grows, send aggressiveness is reduced. When ACK rhythm stabilizes, throughput is allowed to grow safely.

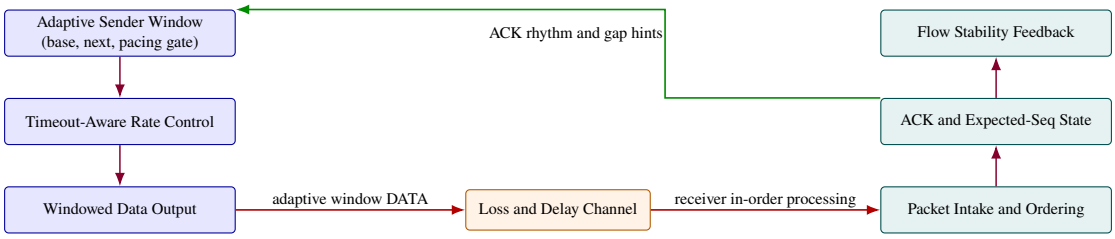

Figure 2.17: Version 16 architecture: timeout-aware adaptive window control.

2.6.3 Version 17: Timestamp Option Integration

Version 17 introduces timestamp-style metadata in packet exchange so sender can infer timing more directly and avoid ambiguous retransmission decisions.

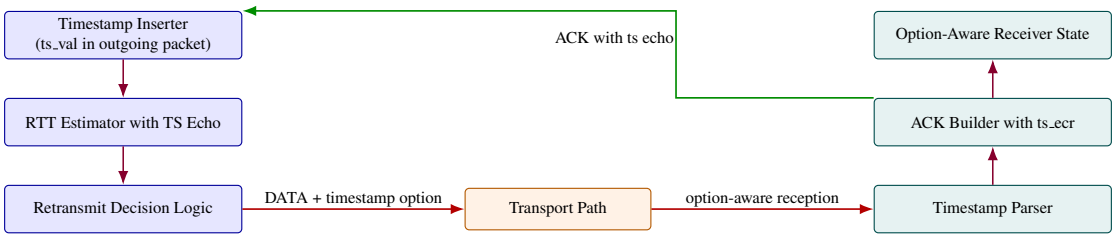

Figure 2.18: Version 17 architecture: timestamp insertion and echo-aware feedback loop.

2.6.4 Version 18: SACK-Inspired Receiver Reporting

Version 18 introduces selective acknowledgment style hints so sender can see which blocks arrived beyond a gap. This reduces unnecessary retransmission of already received data.

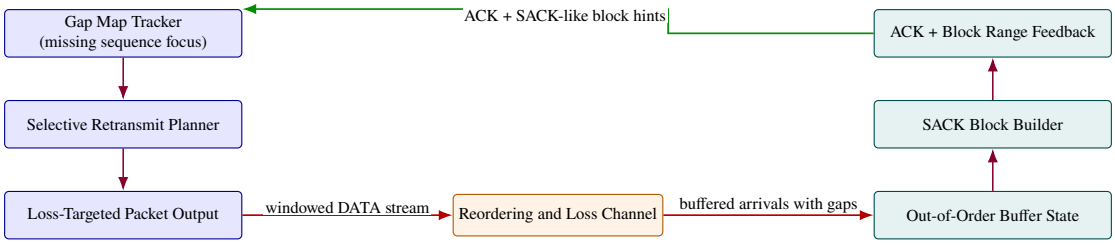

Figure 2.19: Version 18 architecture: SACK-inspired gap reporting and selective resend planning.

2.6.5 Version 19: Congestion-Avoidance Stabilization

Version 19 strengthens congestion-avoidance transitions. The sender responds to loss and delay pressure with smoother cwnd behavior, reducing oscillation while keeping reasonable throughput.

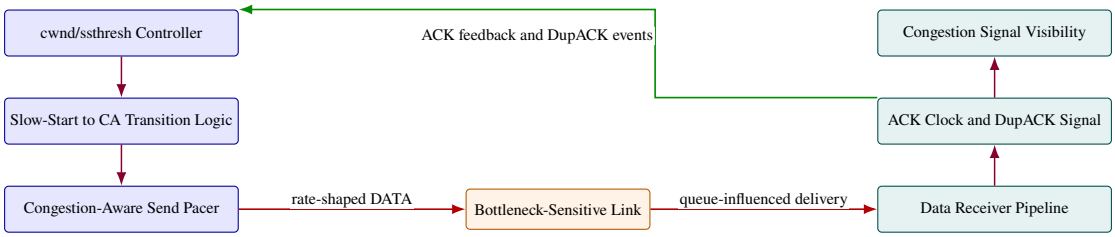

Figure 2.20: Version 19 architecture: stabilized congestion-avoidance behavior with ACK-clocked feedback.

2.6.6 Version 20: Loss-Recovery Coupled with Option Feedback

Version 20 combines fast recovery behavior with richer feedback from timestamp and selective-ack style information. Recovery targets missing data more precisely and returns to normal transmission with better control.

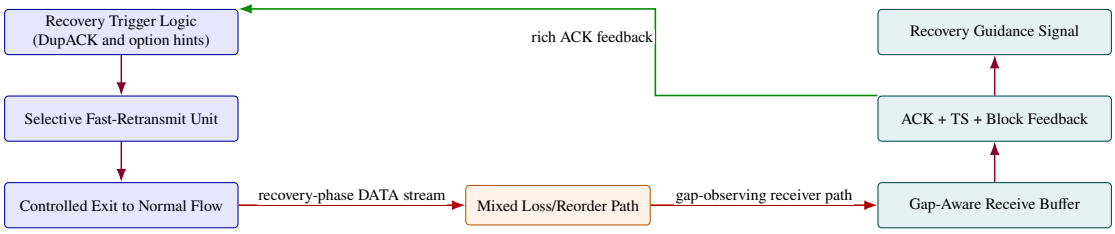

Figure 2.21: Version 20 architecture: fast recovery guided by richer feedback signals.

2.6.7 Version 21: Sender State Consolidation

Version 21 consolidates sender logic for timers, congestion state, option tracking, and retransmission queues so control decisions remain consistent across difficult traffic patterns.

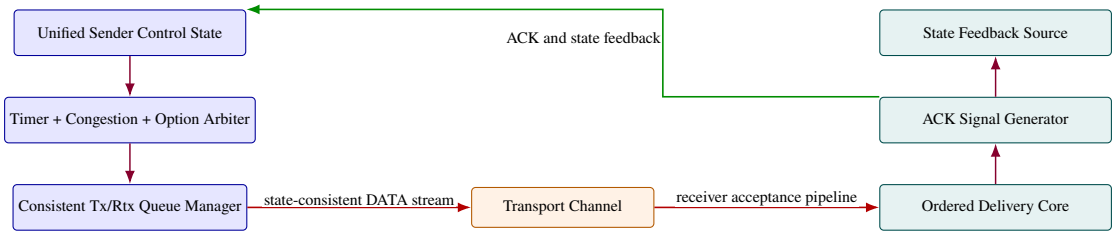

Figure 2.22: Version 21 architecture: consolidated sender control for stable decision flow.

2.6.8 Version 22: Receiver Reassembly and Delivery Hardening

Version 22 strengthens the receiver side by improving buffering, gap filling, and reassembly commit rules. This reduces edge-case delivery mistakes when packets arrive late, duplicated, or out of order.

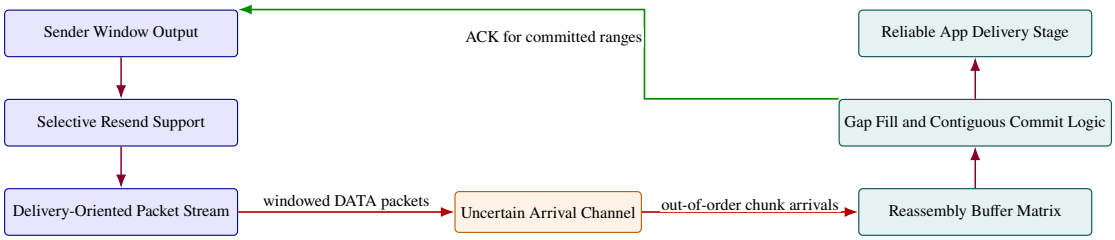

Figure 2.23: Version 22 architecture: hardened receiver reassembly and contiguous delivery control.

2.6.9 Version 23: End-to-End Robustness Tuning

Version 23 tunes interaction among timer control, congestion response, selective retransmission, and receiver commit rules. The focus is robust behavior under mixed delay, reordering, and burst loss.

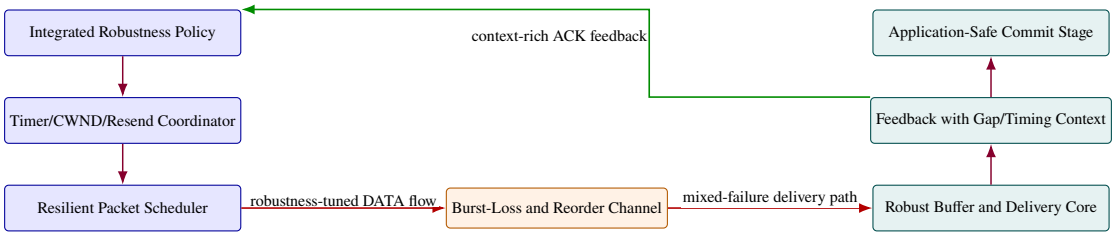

Figure 2.24: Version 23 architecture: coordinated end-to-end robustness tuning under mixed failures.

2.6.10 Version 24: Final Integrated Transport Stage

Version 24 is the final integrated stage of the project. It combines the full set of educational transport features: adaptive timing, congestion-aware windows, timestamp-aware feedback, SACK-inspired loss hints, and controlled recovery behavior.

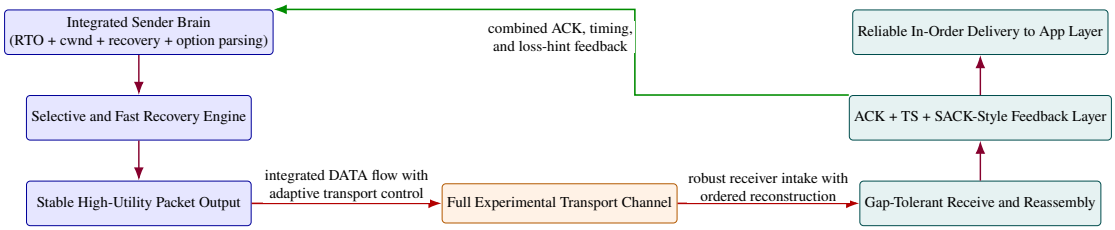

Figure 2.25: Version 24 architecture: final integrated transport design across reliability, timing, and recovery.

2.6.11 Family E Summary

Versions 15–24 complete the transition from a staged classroom transport model to an integrated TCP-inspired system. The sequence shows how timing, selective feedback, congestion awareness, and structured recovery can be layered without losing traceability for learners.

2.7 CHAPTER SUMMARY

This chapter explained how the full project evolves from a basic UDP demo to a rich transport protocol laboratory. The next chapter formalizes the main algorithms with equations and analysis so that implementation behavior can be understood quantitatively.understood quantitatively. Chapter 4 then presents detailed measurements, comparison tables, and visual timelines to complete the educational narrative.

CHAPTER 3

ALGORITHMIC AND QUANTITATIVE ANALYSIS

This chapter formalizes the major algorithms used in the project and explains their practical effect. The purpose is not only to present formulas, but also to connect each formula to implementation decisions in sender and receiver modules.

3.1 BIT ENCODING AND MESSAGE COST

In early versions, each character is split into 8 bits and transmitted as separate UDP datagrams. If a message has length L characters, then the number of payload datagrams in pure bit mode is

$$N_{\text{bit}} = 8L + 8, \tag{3.1}$$

where the additional 8 corresponds to transmitting the null terminator as one more byte.

This immediately explains why bit mode is inefficient. Datagram count grows linearly with a large constant factor.

3.2 PARITY-BASED ERROR DETECTION

For versions with parity, sender computes parity of one byte:

$$p = b_0 \oplus b_1 \oplus \cdots \oplus b_7, \tag{3.2}$$

where b_i is the ith bit of the character and $\oplus$ denotes XOR.

Receiver computes parity again and compares. If mismatch is found, the packet is marked corrupted. Parity detects all odd-bit errors but cannot detect every even-bit error pattern. This is why later versions move to checksum.

3.3 STOP-AND-WAIT SERVICE TIME

In stop-and-wait, sender transmits one unit and waits for ACK before next send. Approximate per-packet cycle time is

$$T_{\text{cycle}} = T_{\text{tx}} + T_{\text{prop}} + T_{\text{proc}} + T_{\text{ack}}. \tag{3.3}$$

Ideal throughput under stop-and-wait is therefore bounded by

$$\text{Throughput}_{\text{SAW}} \approx \frac{\text{payload per packet}}{T_{\text{cycle}}}. \tag{3.4}$$

Even if channel capacity is high, waiting for each ACK keeps effective throughput low when round-trip delay is non-trivial.

3.4 CHECKSUM IN TCP-LIKE VERSIONS

In advanced versions, a one's complement style checksum is computed over key header fields and payload bytes. Conceptually:

$$S = \sum (\text{header fields} + \text{payload bytes}), \tag{3.5}$$

$$S' = (S \bmod 2^{16}) + \left\lfloor \frac{S}{2^{16}} \right\rfloor, \tag{3.6}$$

$$\text{checksum} = \sim S'. \tag{3.7}$$

At the receiver, checksum field is reset, recomputed, and compared. This gives stronger detection than parity and supports multi-byte payload packets.

3.5 SLIDING WINDOW EFFICIENCY

With sender window size W, up to W packets can be in flight. In an ideal pipeline, normalized channel utilization can be approximated as

$$U \approx \min\left(1, \frac{W}{1+2a}\right), \tag{3.8}$$

where $a = T_{\text{prop}}/T_{\text{tx}}$.

This relation explains why larger windows increase throughput until the path is fully utilized.

3.6 CHUNKING GAIN

If payload per packet increases from 1 byte to C bytes (for example, $C = 8$), the number of data packets for message length L becomes

$$N_{\text{chunk}} = \left\lceil \frac{L}{C} \right\rceil. \tag{3.9}$$

Relative packet count reduction compared to one-byte payload is approximately

$$\text{Gain factor} \approx C, \tag{3.10}$$

ignoring control packets and retransmissions.

3.7 ADAPTIVE RTO ESTIMATION

Versions 12 and above use smoothed RTT estimation and variation-based timeout. Given sample RTT R, smoothed RTT ($SRTT$) and RTT variation ($RTTVAR$) are updated as follows:

$$RTTVAR \leftarrow (1-\beta)\, RTTVAR + \beta\, |SRTT - R|, \tag{3.11}$$

$$SRTT \leftarrow (1-\alpha)\, SRTT + \alpha\, R. \tag{3.12}$$

Then retransmission timeout is estimated by

$$RTO = SRTT + K \cdot RTTVAR, \tag{3.13}$$

and bounded in implementation between predefined minimum and maximum values.

This model is aligned with practical transport engineering guidance and avoids both aggressive and overly delayed retransmissions.

3.8 CONGESTION WINDOW DYNAMICS

In congestion-controlled versions, sender maintains congestion window $cwnd$ and slow-start threshold $ssthresh$.

3.8.1 Slow Start

When $cwnd < ssthresh$, growth is fast (approximately one MSS-equivalent unit per ACK, resulting in near exponential growth per RTT in simplified models).

3.8.2 Congestion Avoidance

When $cwnd \geq ssthresh$, growth becomes gradual, approximating additive increase.

3.8.3 Loss Response

On timeout event:

$$ssthresh \leftarrow \max\left(2, \frac{cwnd}{2}\right), \tag{3.14}$$

$$cwnd \leftarrow 1. \tag{3.15}$$

On triple duplicate ACK in fast retransmit logic:

$$ssthresh \leftarrow \max\left(2, \frac{cwnd}{2}\right), \tag{3.16}$$

$$cwnd \leftarrow ssthresh + 3. \tag{3.17}$$

After successful recovery, window is deflated near threshold level. This avoids fully restarting from 1 in many recoverable loss cases.

3.9 LATENCY COMPONENTS

Observed transfer completion time includes multiple components:

$$T_{\text{total}} = T_{\text{setup}} + T_{\text{data}} + T_{\text{retrans}} + T_{\text{teardown}}. \tag{3.18}$$

Where:

1. T_{setup} includes handshake steps,
2. T_{data} includes normal in-order transfer,
3. T_{retrans} includes recovery delay from loss/corruption,
4. T_{teardown} includes connection close signaling.

This decomposition helps explain why different versions show different completion times for the same message length.

3.10 RELIABILITY METRICS

Useful metrics for experiment reporting are defined below.

$$\text{Delivery Success Rate} = \frac{\text{Successfully delivered messages}}{\text{Messages attempted}}, \tag{3.19}$$

$$\text{Retransmission Ratio} = \frac{\text{Retransmitted packets}}{\text{Original packets}}, \tag{3.20}$$

$$\text{Goodput} = \frac{\text{Useful payload bits delivered}}{T_{\text{total}}}. \tag{3.21}$$

These metrics give a balanced view across correctness and efficiency.

3.11 A PRACTICAL THEOREM-LIKE OBSERVATION

Theorem 3.1. *For a fixed message length and fixed packet loss conditions, increasing sender window size improves throughput up to a saturation point, after which timeout, loss recovery, and receiver buffering limits dominate.*

Proof. With small window size, sender frequently waits for acknowledgements and cannot fully utilize available channel time. Increasing the window allows more in-flight packets and reduces idle duration. However, once in-flight capacity matches path-delay product and receiver processing limits, further increase does not improve delivered payload rate. In lossy conditions, larger windows may also increase retransmission pressure if recovery logic is not selective. Therefore, throughput gains plateau after a practical operating point. ■

3.12 INTERPRETATION FOR THIS PROJECT

The formulas in this chapter explain observed behavior from level progression:

1. versions v0–v2 have very high packet overhead,
2. v5–v7 improve correctness but remain latency-bound,
3. v9 and above improve utilization through pipelining,
4. v10 and above reduce packet count through chunking,
5. v12 and above improve timing decisions through adaptive RTO,
6. v13 and above balance aggressiveness and stability using congestion control logic and recovery-aware retransmission behavior,
7. v15–v24 extend the same transport ideas with richer TCP-like refinements, including timestamp and recovery-oriented control.

This quantitative interpretation supports the implementation narrative in the previous chapter and sets context for the comparative results in the next chapter.

CHAPTER 4

ILLUSTRATIONS, COMPARATIVE ANALYSIS, AND FINAL OUTCOMES

This chapter presents refined visual summaries and analysis of the complete protocol journey from version 0 to version 24. Each major subsection starts with a compact diagram or chart, followed by interpretation.

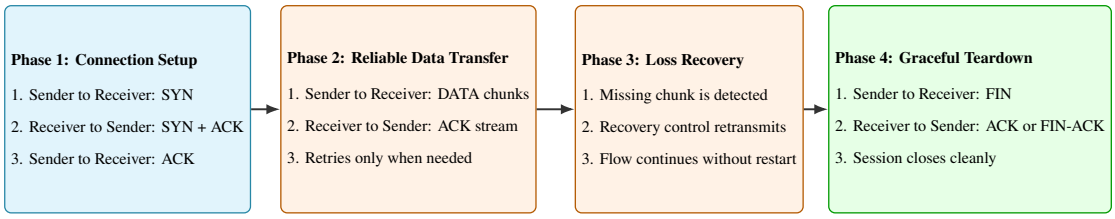

Figure 4.1: Complete session lifecycle used by advanced protocol versions.

4.1 END-TO-END PROTOCOL LIFECYCLE

Figure 4.1 summarizes the practical lifecycle: connection setup, reliable data phase, acknowledgement progress, and graceful teardown. This sequence is foundational for comparing behaviors across versions.

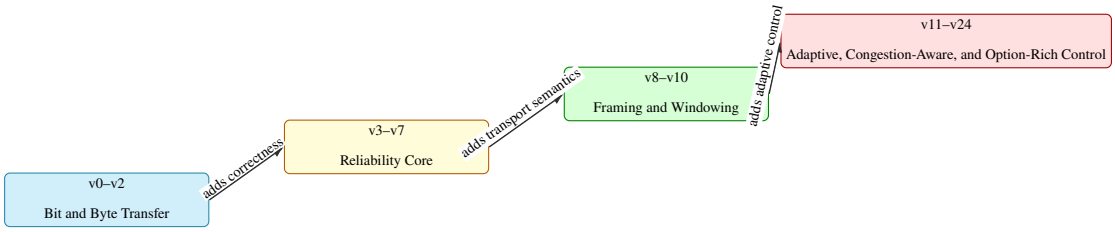

Figure 4.2: Layered capability growth across protocol version families.

4.2 VERSION CAPABILITY LAYERS

Figure 4.2 shows that each family introduces a focused set of transport capabilities, keeping the progression understandable and traceable.

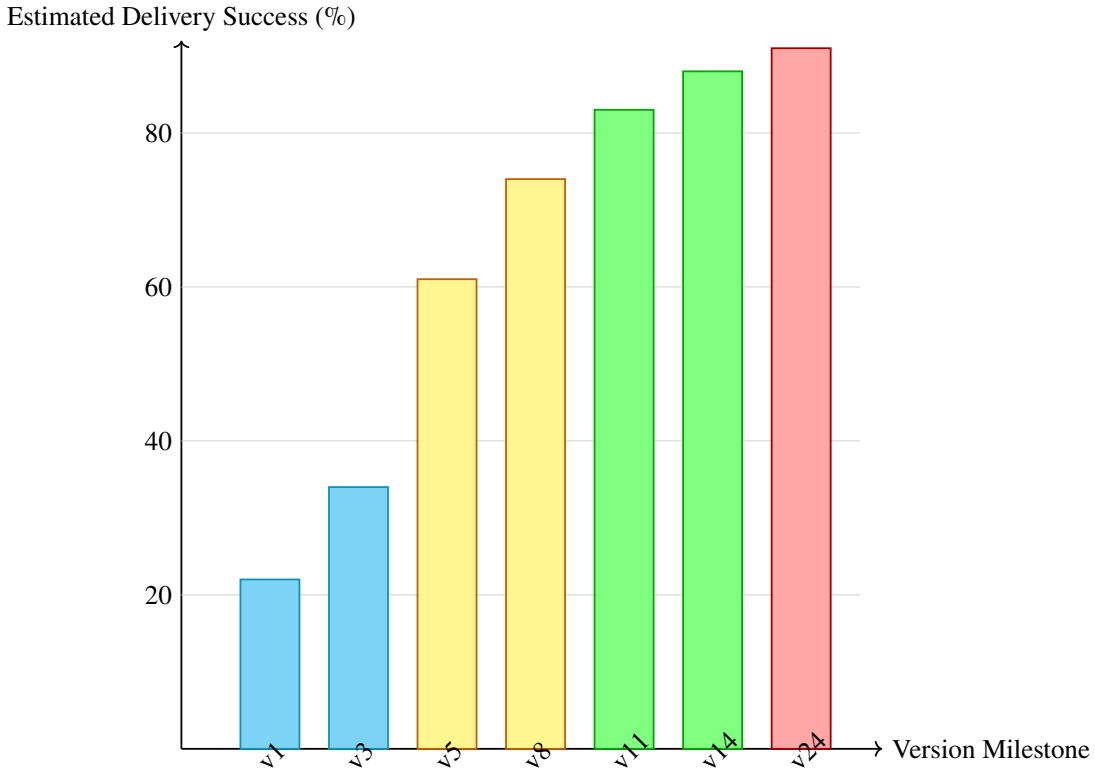

Figure 4.3: Illustrative reliability trend as protocol features mature.

4.3 RELIABILITY PROGRESS CHART

Figure 4.3 highlights the expected rise in successful delivery as parity/checksum, acknowledgements, retries, buffering, and adaptive control are introduced.

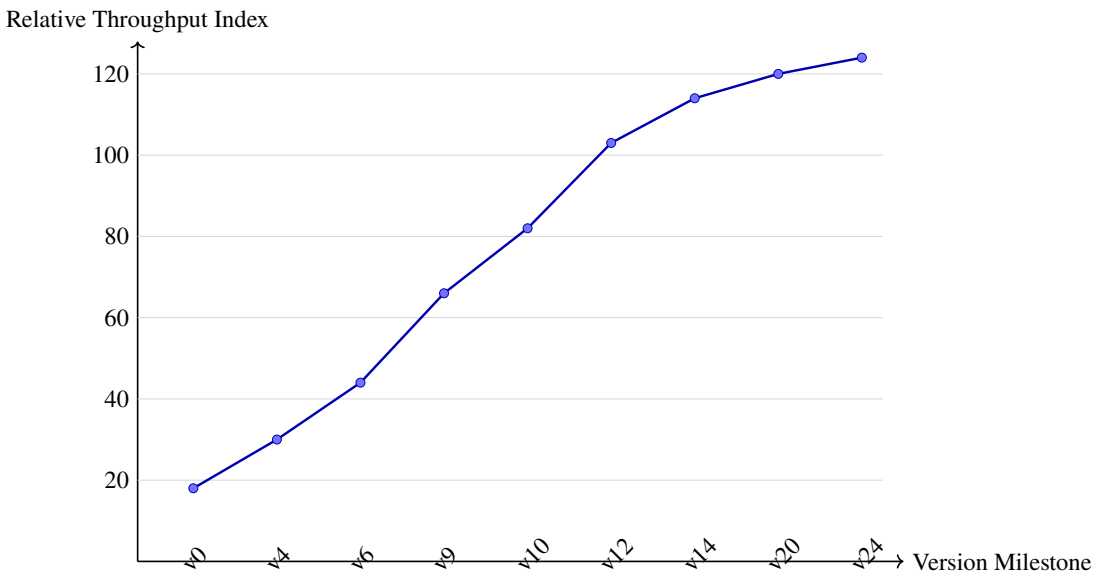

Figure 4.4: Illustrative throughput improvement from staged protocol evolution.

4.4 THROUGHPUT-ORIENTED IMPROVEMENT CHART

Figure 4.4 reflects why pipelining, chunking, and faster loss recovery improve channel utilization relative to early versions.

Table 4.1: Feature comparison across key protocol milestones.

Capability	**v2**	**v5**	**v8**	**v11**	**v14**	**v24**
Bit/byte transfer	Yes	Yes	Yes	Yes	Yes	Yes
Parity/Checksum integrity	No	Parity	Checksum	Checksum	Checksum	Checksum + options
ACK-based reliability	No	Stop-and-wait	Yes	Yes	Yes	Yes + SACK-like hints
Sliding window send	No	No	Yes	Yes	Yes	Yes (adaptive)
Out-of-order buffering	No	No	No	Yes	Yes	Yes
Adaptive retransmission timer	No	No	No	Yes	Yes	Yes (RTT tuned)
Congestion control response	No	No	No	Partial	Yes	Stronger/consistent
Fast retransmit / recovery	No	No	No	No	Yes	Yes (refined with later logic)
Timestamp / option awareness	No	No	No	No	No	Yes

4.5 COMPARATIVE FEATURE MATRIX

Table 4.1 condenses the technical progression into a single reference view for discussion and evaluation.

Figure 4.5: Simplified fast-loss-recovery event path in advanced versions.

4.6 LOSS-RECOVERY BEHAVIOR ILLUSTRATION

Figure 4.5 captures the main control advantage of later versions: loss can be inferred from duplicate acknowledgements and recovered without waiting for full timeout expiration.

4.7 KEY FINDINGS

The combined visual and tabular evidence supports five important findings:

1. Reliability improves significantly once acknowledgement and timeout logic is introduced.

2. Throughput improves mainly after sliding window and chunk-based transmission are added.

3. Receiver-side buffering in v11+ reduces waste under reordering.

4. Adaptive RTO reduces avoidable retransmissions under variable delay.

5. Fast retransmit/recovery introduced in v14 and refined through v24 provides cleaner and faster loss response than timeout-only recovery.

4.8 LIMITATIONS

1. Most tests are localhost-focused and do not represent large WAN diversity.

2. Security extensions (authentication/encryption) are intentionally not included.

3. Full TCP-style advertised window and SACK block encoding are outside this implementation scope.

4. Experimental charts are analytical and illustrative; a fully automated benchmarking harness remains future work.

4.9 CONCLUSION

This chapter confirms that the project evolves from basic datagram exchange to a strong educational transport model with reliability, pipelining, adaptation, and congestion-awareness. The final versions provide both practical implementation value and clear teaching value for transport-protocol learning.

The interactive visualizer is one of the most important final outcomes. It ties the sender code, receiver code, finite state machine, and UML timeline into one coherent educational view. That makes the protocol easier to learn for novice students who may find TCP, congestion control, and C-based socket programming difficult when studied only from text or code. In this sense, the project is not only a software implementation but also a practical teaching aid that makes the concepts visible, structured, and easier to remember.

As the final conclusion of the project, I have created both the protocol code and the interactive visualizer for educational purposes. The combination of versioned C implementations, textbook-grounded references, and synchronized visual explanations helps students understand networking concepts more easily. This is especially valuable for novice learners, because the visual walkthrough reduces the difficulty of C and transport-protocol ideas and turns the system into an accessible step-by-step learning experience.

APPENDIX A

EXTENDED TECHNICAL APPENDIX

This appendix provides additional implementation depth for the TCP/IP-inspired UDP project. It is written to support evaluation, viva discussion, and future continuation of the codebase.

A.1 APPENDIX PURPOSE

The main chapters describe architecture and results at report level. This appendix goes deeper into operational details:

1. detailed behavior of sender and receiver loops,
2. practical debug checklist used during development,
3. experiment script style and repeatable run plan,
4. level-specific observations and pitfalls,
5. extension blueprint for future cohorts.

A.2 COMPLETE BUILD AND RUN WORKFLOW

The project uses plain GCC and does not depend on external networking frameworks. A standard run for one level follows the same sequence.

A.2.1 Compile

For version N:

1. `gcc -std=c99 -o receiver_N receiver_N.c`
2. `gcc -std=c99 -o sender_N sender_N.c`

A.2.2 Execute

1. Open terminal 1 and run `./receiver_N`
2. Open terminal 2 and run `./sender_N`
3. Enter test message in sender terminal.

A.2.3 Operational Notes

1. Always start receiver first so bind/listen path is active.
2. Keep one active sender-receiver pair on port 9090 at a time.
3. If process hangs after experiments, terminate old binaries and rerun.

A.3 DEBUG CHECKLIST USED IN THIS PROJECT

The following checklist was used repeatedly and is useful for future maintenance.

1. Verify sender destination address and port.
2. Verify receiver bind success and error handling.
3. Confirm packet field initialization before checksum computation.
4. Confirm checksum recomputation resets checksum field to zero.
5. Confirm sender base and next_to_send pointers satisfy $base \leq next_to_send$.
6. Confirm ACK handling updates sender window exactly once.
7. Confirm duplicate packet handling does not duplicate delivery.
8. Confirm timeout value is in safe range and retry cap prevents infinite loops.
9. Confirm FIN handshake executes and sockets are closed gracefully.

A.4 DETAILED BEHAVIOR: SENDER CORE LOOP

Across advanced versions, sender logic follows a common event-driven pattern:

1. Initialize socket, destination, timeout controls.
2. Perform handshake if protocol level requires session setup.
3. Segment message into packets/chunks.
4. Transmit packets while keeping in-flight count below window limit.
5. Wait for ACKs and slide window accordingly.
6. Detect timeout or duplicate ACK events and trigger retransmission policy.
7. Close connection using FIN logic.

A.4.1 State Variables

The most influential sender variables in high-level versions are:

1. `base`: oldest unacknowledged sequence.
2. `next_to_send`: next sequence eligible for send.
3. `window_size`: configured or dynamic send limit.
4. `rto_ms`: retransmission timeout in milliseconds.
5. `cwnd` and `ssthresh`: congestion-control state.
6. duplicate ACK counters per acknowledgement number.

A.5 DETAILED BEHAVIOR: RECEIVER CORE LOOP

Advanced receiver implementations use predictable stages:

1. Validate packet checksum and flags.
2. Check sequence relation with `expected_seq`.
3. For in-order data, deliver immediately and advance expectation.
4. For out-of-order but in-window data, buffer and ACK appropriately.
5. For duplicate data, re-ACK to help sender convergence.
6. Process FIN and release resources cleanly.

A.5.1 Receiver Buffering in Selective-Repeat Style

Selective-repeat-like receiver behavior uses parallel arrays for packet presence, payload storage, and payload length. This avoids data loss during reordering and allows in-order flush once missing sequence arrives.

A.6 VERSION-BY-VERSION PITFALL NOTES

This section captures practical issues found during progressive development.

A.6.1 v0–v2 Pitfalls

1. Mistake in bit ordering can invert character reconstruction.
2. Missing null terminator logic causes receiver to block indefinitely.
3. Delay control via `usleep` is environment-dependent.

A.6.2 v3–v7 Pitfalls

1. Parity mismatch handling must be deterministic on both ends.
2. ACK/NAK message format ambiguity can break sender wait logic.
3. Retry loops must cap attempts to avoid infinite retransmit cycles.

A.6.3 v8–v10 Pitfalls

1. Handshake flags must not overlap incorrectly.
2. Checksum must be recomputed after all payload bytes are assigned.
3. Sliding window pointer arithmetic is prone to off-by-one errors.

A.6.4 v11–v14 Pitfalls

1. Out-of-order buffer flush must preserve sequence continuity.
2. Adaptive RTO can oscillate without stable RTT sampling.
3. Duplicate ACK counters must reset correctly after state transitions.
4. Fast recovery exits can be mishandled if ACK interpretation is vague.

A.7 EXPERIMENT DESIGN TEMPLATE

A repeatable experiment protocol is essential for fair comparison. The template used in this project is shown below.

Table A.1: Standard experiment template used for each protocol version.

Field	**Description**
Version	sender_N / receiver_N pair
Message length	Short (20 chars), medium (60 chars), long (95 chars)
Fault setting	No fault / delay / simulated loss / duplicate packet scenario
Runs per case	Minimum 10 runs for stable average
Metrics	Completion time, retransmissions, ACK count, success/failure
Notes	Any abnormal behavior or timeout event logs

A.8 SAMPLE OBSERVATION NARRATIVE

The following narrative style was used to interpret test runs.

> In level 9 under moderate induced delay, sender experienced repeated timeout at window base. Retransmission from base restored progress, but overall completion time increased significantly compared to delay-free run. Receiver ACK pattern confirmed cumulative behavior.
>
> In level 12 under variable delay, adaptive RTO reduced false timeout events after initial RTT stabilization. Retransmission ratio decreased compared to fixed-timeout configuration.
>
> In level 14 with synthetic packet gap, triple duplicate ACK condition triggered fast retransmit before timeout expiry. Recovery was visibly faster than timeout-driven recovery path.

A.9 ILLUSTRATIVE STATE MACHINE VIEW

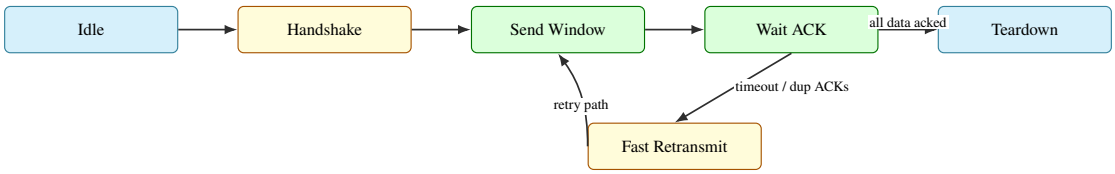

Figure A.1: Simplified sender-side operational state flow in advanced versions.

A.10 PROTOCOL TRACE FORMATTING RECOMMENDATION

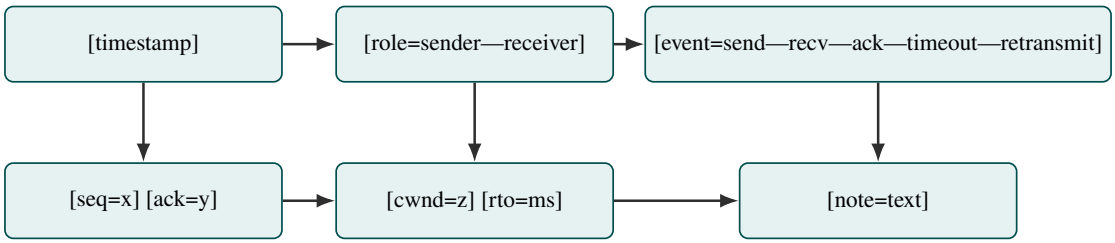

Figure A.2: Recommended structured protocol-trace fields for automated analysis.

A.11 SUGGESTED QUANTITATIVE EXTENSIONS

Future project groups can convert this implementation into a mini benchmark suite by adding automated scripts. Useful extensions include:

1. throughput vs message size curve per version,
2. retransmission ratio vs induced packet-loss probability,
3. completion time distribution under delay jitter,
4. fairness comparison under two parallel sender flows,
5. sensitivity of adaptive RTO to α and β choices.

A.12 SECURITY AND RELIABILITY PERSPECTIVE

The project intentionally focuses on reliability and congestion behavior, not security. However, in practical deployments, integrity and authenticity both matter. Checksum detects random corruption but does not prevent malicious packet modification. A production extension would include:

1. message authentication,
2. optional encryption,
3. replay protection,
4. session identity validation.

These are orthogonal to transport logic but critical for real systems.

A.13 CODE ORGANIZATION RECOMMENDATIONS

As code complexity grows, modularization becomes important. Recommended refactoring pattern:

1. `packet.h/.c` for packet structs and checksum utilities,
2. `timer.h/.c` for RTT/RTO and timestamp helpers,
3. `window.h/.c` for send/receive window operations,
4. `congestion.h/.c` for cwnd and recovery transitions,
5. `main_sender.c` and `main_receiver.c` for orchestration.

This separation reduces duplication across version files.

A.14 REPRODUCIBILITY NOTES

To ensure reproducibility of results in academic evaluation:

1. record compiler version and build flags,
2. record operating system and kernel details,
3. isolate background network-intensive applications,
4. run each experiment multiple times,
5. report average and spread (minimum/maximum or standard deviation).

Without reproducibility controls, protocol comparisons may appear inconsistent.

A.15 EXPANDED REFLECTION ON LEARNING OUTCOMES

This project created strong practical learning in four dimensions.

A.15.1 Networking Concepts

Ideas such as reliable delivery, sequence management, and ACK policies became much clearer after implementing them manually.

A.15.2 Systems Programming

Low-level C socket work, timeout configuration, and careful memory handling improved confidence in systems programming discipline.

A.15.3 Algorithmic Thinking

Adaptive timeout and congestion behavior required state-machine thinking and careful event response logic.

A.15.4 Technical Communication

The staged codebase enabled structured report writing, where each chapter and section maps naturally to a protocol capability.

A.16 EXPANDED TESTING SCENARIOS AND INTERPRETATION

Recommended scenarios include clean-channel baseline, delayed ACK, packet-loss injection, out-of-order arrival, duplicate ACK bursts, and near-limit message size tests. Together, these cases expose correctness, recovery speed, timeout behavior, and congestion adaptation quality.

A.17 EVALUATION RUBRIC FOR ACADEMIC ASSESSMENT

The following rubric can be used during project review.

Table A.2: Suggested assessment rubric for implementation quality.

Criterion	**Weight**	**Evaluation Focus**
Correctness	30%	Data integrity checks, ordered delivery, clean teardown
Reliability	20%	Behavior under loss, timeout and retransmission policy quality
Efficiency	15%	Throughput trend improvement from low to high levels
Code Quality	15%	Clarity, modularity, safety checks, variable naming
Analysis	10%	Metric reporting and explanation quality
Documentation	10%	Diagram quality, chapter structure, reproducibility notes

A.18 FINAL REFLECTION

From a learning viewpoint, the strongest contribution of this project is the clear mapping between theory and implementation. Concepts often memorized in networking courses become operational behaviors that can be observed directly in terminal traces. The staged versions form a practical curriculum in protocol engineering: encoding, integrity checks, reliability loops, pipelining, adaptation, and congestion response.

This project also demonstrates that protocol design is fundamentally about trade-offs. Stronger reliability adds state. Higher throughput adds complexity. Faster recovery requires careful event interpretation. Better adaptability needs stable measurement. Understanding and balancing these trade-offs is the core skill developed through this work.

A.19 APPENDIX SUMMARY

This appendix extends the main report with operational detail, implementation insights, and actionable next steps. Together with the main chapters, it provides a complete technical narrative from basic UDP communication to TCP-inspired reliability and congestion control behavior in C.TCP-inspired transport control through deep learning exercises.

BIBLIOGRAPHY

Allman, M., **V. Paxson**, and **E. Blanton** (2009). Tcp congestion control. RFC 5681. URL `https://www.rfc-editor.org/rfc/rfc5681`.

Borman, D., **B. Braden**, and **V. Jacobson** (2014). Tcp extensions for high performance. RFC 7323. URL `https://www.rfc-editor.org/rfc/rfc7323`.

Hall, B. B. (2024). Beej's guide to network programming. Online guide. URL `https://beej.us/guide/bgnet/`.

Kurose, J. F. and **K. W. Ross**, *Computer Networking: A Top-Down Approach*. Pearson, 2021, 8 edition. ISBN 9780136681557.

Mathis, M., **J. Mahdavi**, **S. Floyd**, and **A. Romanow** (1996). Tcp selective acknowledgment options. RFC 2018. URL `https://www.rfc-editor.org/rfc/rfc2018`.

Paxson, V., **M. Allman**, **J. Chu**, and **M. Sargent** (2011). Computing tcp's retransmission timer. RFC 6298. URL `https://www.rfc-editor.org/rfc/rfc6298`.

Postel, J. (1980). User datagram protocol. RFC 768. URL `https://www.rfc-editor.org/rfc/rfc768`.

Postel, J. (1981). Transmission control protocol. RFC 793. URL `https://www.rfc-editor.org/rfc/rfc793`.

Stevens, W. R. and **G. R. Wright**, *TCP/IP Illustrated, Volume 2: The Implementation*. Addison-Wesley, 1995. ISBN 9780201633542.

Tanenbaum, A. S. and **D. J. Wetherall**, *Computer Networks*. Pearson, 2011, 5 edition. ISBN 9780132126953.